PayPal JunoDB: High-Level Architecture & System Design

Stateless Go Proxies, The 2D Storage Grid, Synchronous Quorums & Multi-AZ Zero Downtime

SYSTEM ARCHITECTURE
CORE INFRASTRUCTURE

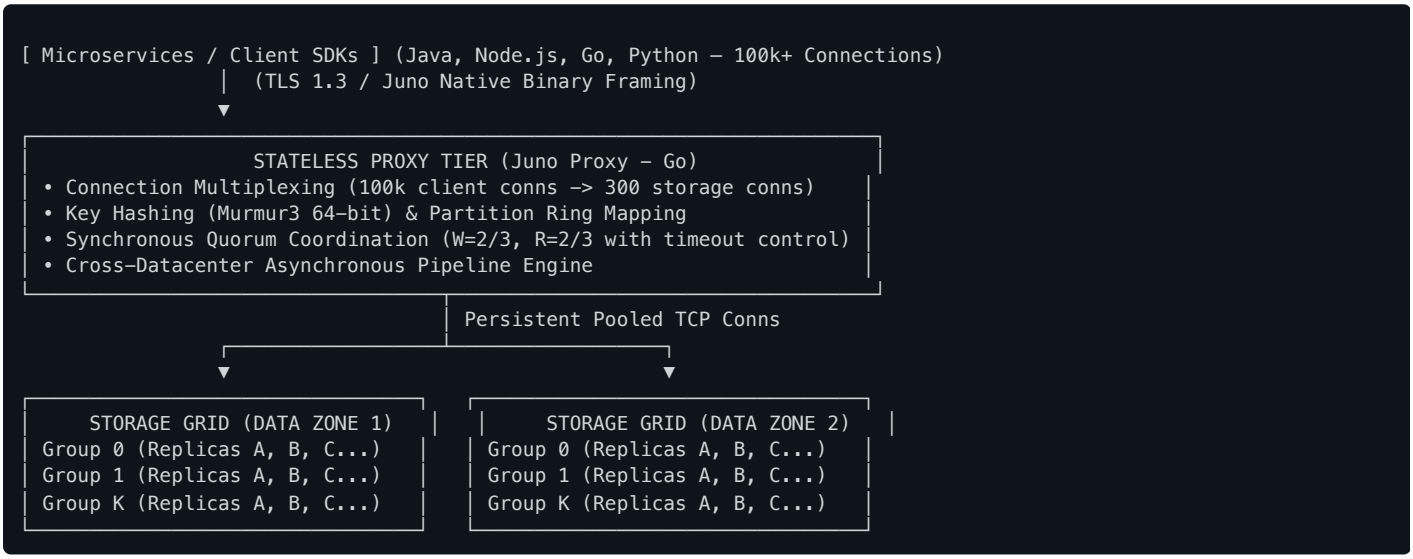
1. Executive Summary & Design Philosophy

JunoDB is PayPal's ultra-reliable, petabyte-scale distributed key-value store serving over **350 billion requests daily** across thousands of critical microservices. Unlike off-the-shelf distributed caches like Redis Cluster or Memcached that sacrifice consistency or multi-tenant resilience for pure in-memory raw speed, JunoDB was purpose-engineered to survive catastrophic multi-datacenter disasters without financial data corruption or stale session reads.

Core Engineering Trade-Off: Zero Data Loss vs. Asynchronous Speed
Standard Redis deployments rely on asynchronous primary-to-replica replication. In the event of an ungraceful master failure or network partition, un-replicated write buffers are permanently lost. For financial authentication tokens, two-factor challenges, and payment idempotency records, data loss triggers double-charges or unauthorized access. JunoDB enforces **synchronous quorum writes ($W \ge 2$ out of $N=3$)** at sub-millisecond latencies.

2. Tiered System Topology: Decoupling Compute from Storage

JunoDB strictly decouples stateless connection handling from stateful storage engines. The architecture comprises three foundational tiers:



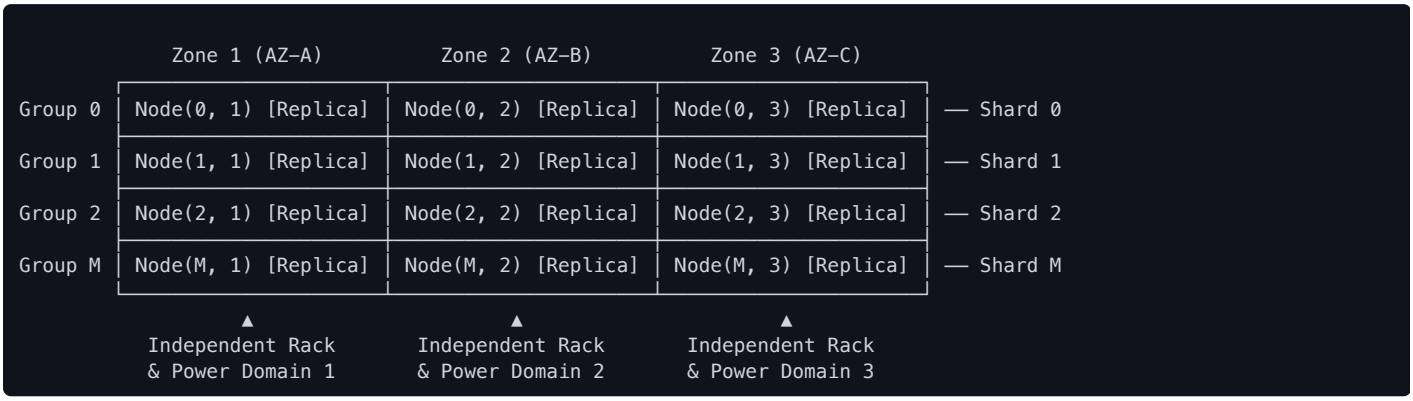
3. Connection Multiplexing & Proxy Layer Mechanics

At PayPal scale, tens of thousands of microservice instances open connections simultaneously. Direct connections to storage nodes would trigger thread exhaustion and TCP connection state thrashing. The Go-based Proxy solves this:

- **Connection Funneling:** Over 100,000 inbound client TCP sockets terminate at the Proxy layer. The proxy multiplexes these requests across a fixed, pre-warmed pool of several hundred persistent connections to the backend storage nodes.
- **Deterministic Shard Routing:** The proxy hashes the incoming record key (K) using 64-bit MurmurHash3 to map the record directly to a Partition Shard ID without distributed consensus overhead.
- **Quorum Fan-Out:** Upon receiving a write request, the proxy simultaneously issues three network writes to the three designated storage replicas in the partition group, returning a 200 OK as soon as two replicas acknowledge.

4. The 2D Storage Grid: Orthogonal Shards & Failure Zones

To achieve six nines (99.9999%) availability, JunoDB arranges storage nodes into a **2D Storage Grid**. In this matrix, rows represent *Storage Groups* (shards) and columns represent *Failure Zones* (independent Availability Zones or power/rack failure domains):



Why this topology is bulletproof: If an entire physical datacenter or AWS Availability Zone (e.g., Zone 1) suffers a complete network partition or power outage, *every single shard in the cluster* retains exactly two live replicas (in Zone 2 and Zone 3). Because JunoDB operates on a 2-of-3 synchronous quorum ($W=2, R=2$), the entire database continues operating at 100% capacity with **zero read/write degradation and zero failover latency**.

5. Synchronous Quorum Math vs. Competing Distributed Engines

System	Replication Model	Quorum Config	Failover Behavior	Data Loss Risk on Crash
JunoDB	Synchronous Multi-Zone	N=3, W=2, R=2	Zero failover delay; immediate surviving quorum	**Zero** (Acknowledged writes survive AZ loss)
Redis Cluster	Asynchronous Master/Replica	Primary ACK only	Sentinel / Raft failover takes 3–15 seconds	**High** (In-flight replication buffer dropped)
Apache Cassandra	Async / Tunable Paxos	N=3, Local Quorum	Decentralized gossip, eventual consistency	Low, but vulnerable to read-repair skew
Amazon DynamoDB	Multi-Paxos Synchronous	N=3, W=2, R=2	Leader lease election handled internally	**Zero** (Closed source proprietary service)

6. Pluggable Storage Engine Backends

Ephemeral Memory Engine (Redis Replacement)
Optimized for high-throughput, short TTL use cases (OAuth tokens, risk scores, web sessions). Employs an in-memory hash table with generational arena allocators, eliminating GC pauses and bypassing disk I/O entirely. Sub-millisecond p99 latencies.

Persistent RocksDB Engine (LSM-Tree on NVMe)
Optimized for long-lived records requiring guaranteed durability across node reboots (payment idempotency keys, merchant configurations). Uses Log-Structured Merge-trees with direct I/O and Bloom filters on local NVMe SSD drives.

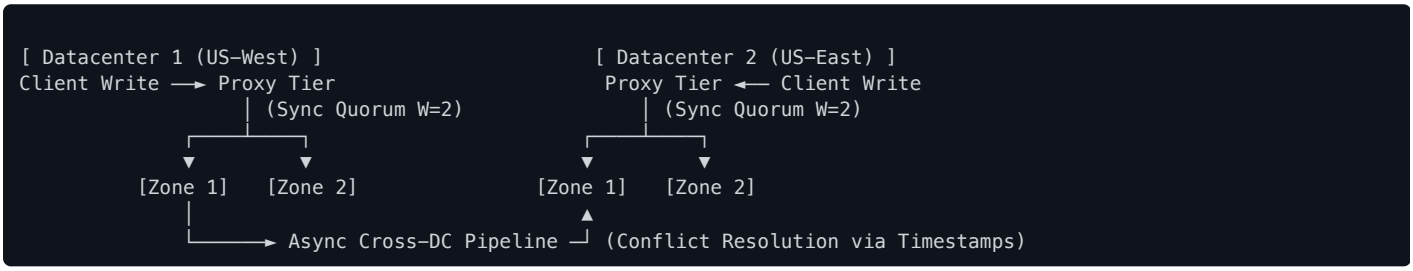
7. Failure Handling & Network Partition Tolerance

In distributed financial environments, partial node failures, slow disks ("gray failures"), and network partitions are normal operational conditions. JunoDB handles them gracefully:

- **Node Crash / Network Drop:** If Replica 1 fails during a write, the proxy still receives successful ACKs from Replica 2 and Replica 3. The write succeeds instantly without blocking. The failed node catches up upon recovery via cluster state gossip and anti-entropy reconcilers.
- **Split-Brain Elimination:** Because write quorum ($W=2$) requires a strict majority of $N=3$, two partitioned network segments can never independently form a quorum ($2 + 2 > 3$). Split-brain write divergent states are mathematically impossible.
- **Hedging & Tail Latency Truncation:** For read operations ($R=2$), the proxy can initiate hedged reads—querying the fastest two replicas and canceling the redundant query, shielding callers from garbage collection spikes on individual storage nodes.

8. Active-Active Cross-Datacenter Asynchronous Pipeline

While intra-datacenter writes are strictly synchronous across three availability zones, cross-datacenter (e.g., US-West to US-East) writes are propagated asynchronously via an integrated pipeline to bypass speed-of-light inter-region latency (60ms+):



9. Key Architectural Lessons for High-Reliability Systems

Architectural Rules Distilled from PayPal's JunoDB

- **Separate Stateless Proxying from Stateful Sharding:** Moving connection multiplexing, client protocol serialization, and quorum logic out of storage nodes keeps the storage engine simple, lean, and resilient against memory fragmentation.
- **Design Topologies Orthogonal to Infrastructure Failure Domains:** Arranging replicas along independent power/rack/AZ boundaries converts catastrophic datacenter disasters into simple single-node failovers with zero operational impact.
- **Synchronous Quorums are Non-Negotiable for Financial Scale:** Never rely on asynchronous master-slave replication when money, user authentication tokens, or idempotency keys are on the line. Mathematical quorum guarantees eliminate the nightmare of silent data divergence.
- **Tail Latency is Controlled by Quorum Over-Provisioning:** By reading from any 2 out of 3 replicas, JunoDB dynamically routes around slow, degraded, or GC-stalled nodes, keeping p99.9 latencies rock-solid at sub-millisecond thresholds.